

```

import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras.datasets import mnist
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout, BatchNormalization
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.optimizers import Adam
from tensorflow.keras import regularizers

(X_train, y_train), (X_test, y_test) = mnist.load_data()
X_train = X_train.reshape(60000, 784).astype('float32') / 255.0
X_test = X_test.reshape(-1, 784).astype('float32') / 255.0
y_train_cat = to_categorical(y_train)
y_test_cat = to_categorical(y_test)

# — No Regularization (Baseline) —————
def model_baseline():
    m = Sequential()
    m.add(Dense(256, input_dim=784, activation='relu'))
    m.add(Dense(128, activation='relu'))
    m.add(Dense(10, activation='softmax'))
    return m

# — L1 Regularization —————
def model_l1():
    m = Sequential()
    m.add(Dense(256, input_dim=784, activation='relu',
                kernel_regularizer=regularizers.l1(0.001)))
    m.add(Dense(128, activation='relu',
                kernel_regularizer=regularizers.l1(0.001)))
    m.add(Dense(10, activation='softmax'))
    return m

# — L2 Regularization —————
def model_l2():
    m = Sequential()
    m.add(Dense(256, input_dim=784, activation='relu',
                kernel_regularizer=regularizers.l2(0.001)))
    m.add(Dense(128, activation='relu',
                kernel_regularizer=regularizers.l2(0.001)))
    m.add(Dense(10, activation='softmax'))
    return m

# — Dropout Regularization —————
def model_dropout():
    m = Sequential()
    m.add(Dense(256, input_dim=784, activation='relu'))
    m.add(Dropout(0.4))
    m.add(Dense(128, activation='relu'))
    m.add(Dropout(0.3))
    m.add(Dense(10, activation='softmax'))
    return m

# — Batch Normalization —————
def model_batchnorm():
    m = Sequential()
    m.add(Dense(256, input_dim=784, activation='relu'))
    m.add(BatchNormalization())
    m.add(Dense(128, activation='relu'))
    m.add(BatchNormalization())
    m.add(Dense(10, activation='softmax'))
    return m

models_dict = {
    'No Regularization': model_baseline(),
    'L1 Regularization': model_l1(),
    'L2 Regularization': model_l2(),
    'Dropout': model_dropout(),
    'Batch Norm': model_batchnorm()
}

histories = {}
results = {}

for name, model in models_dict.items():
    print(f"\nTraining: {name}")
    model.compile(optimizer=Adam(0.001),
                  loss='categorical_crossentropy',
                  metrics=['accuracy'])
    h = model.fit(X_train, y_train_cat,
                  batch_size=128, epochs=10

```

```

        batch_size=128, epochs=10,
        validation_split=0.1, verbose=0)
    histories[name] = h
    test_acc = model.evaluate(X_test, y_test_cat, verbose=0)[1]
    results[name] = test_acc
    print(f" Test Accuracy: {test_acc*100:.2f}%")

# Plot
plt.figure(figsize=(14, 5))
plt.subplot(1, 2, 1)
for name, h in histories.items():
    plt.plot(h.history['val_accuracy'], label=name)
plt.title('Validation Accuracy – Regularization Comparison')
plt.xlabel('Epoch'); plt.ylabel('Accuracy'); plt.legend(fontsize=8)

plt.subplot(1, 2, 2)
for name, h in histories.items():
    plt.plot(h.history['val_loss'], label=name)
plt.title('Validation Loss – Regularization Comparison')
plt.xlabel('Epoch'); plt.ylabel('Loss'); plt.legend(fontsize=8)

plt.tight_layout(); plt.show()

print("\n— Final Test Accuracy —")
for name, acc in results.items():
    print(f" {name:22s}: {acc*100:.2f}%")

# Bar graph for final test accuracy (Regularization comparison)
plt.figure()
names = list(results.keys())
accuracies = list(results.values())

plt.bar(names, accuracies)
plt.title('Final Test Accuracy – Regularization Techniques')
plt.xlabel('Technique')
plt.ylabel('Accuracy')
plt.ylim(0, 1)

# Rotate labels for better readability
plt.xticks(rotation=20)

# Add values on top of bars (important for reports)
for i, v in enumerate(accuracies):
    plt.text(i, v + 0.01, f"{v*100:.2f}%", ha='center', fontsize=8)

plt.tight_layout()
plt.show()

```

Downloading data from <https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz>

11490434/11490434 — 0s 0us/step

/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106: UserWarning: Do not pass an `input\_shape` to `input`
super().__init__(activity_regularizer=activity_regularizer, **kwargs)

Training: No Regularization

Test Accuracy: 97.88%

Training: L1 Regularization

Test Accuracy: 94.71%

Training: L2 Regularization

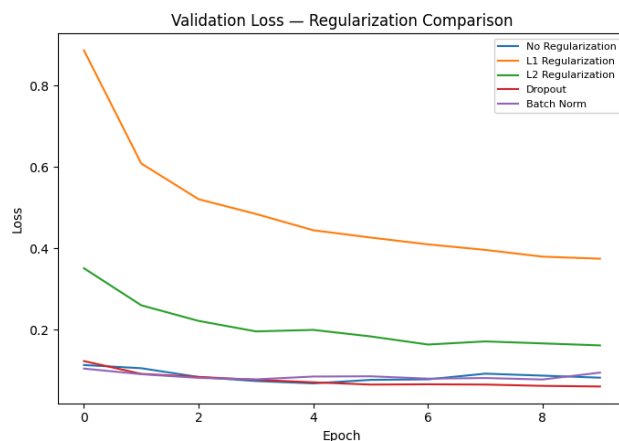
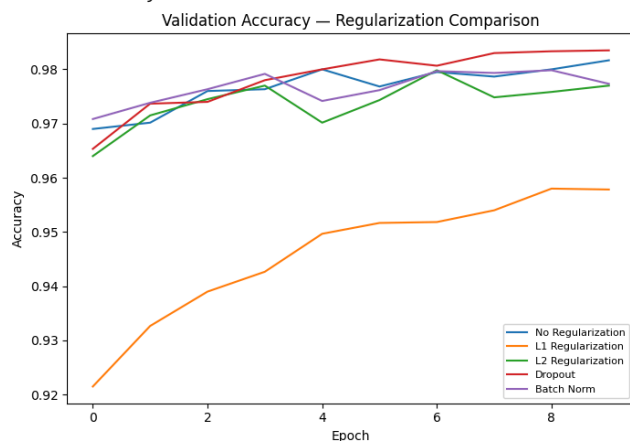
Test Accuracy: 97.33%

Training: Dropout

Test Accuracy: 97.99%

Training: Batch Norm

Test Accuracy: 97.61%



— Final Test Accuracy —

No Regularization : 97.88%

L1 Regularization : 94.71%

L2 Regularization : 97.33%

Start coding or [generate](#) with AI.

